

AI & IT-Security (WS25/26)

Niklas Rodenbüsch | February 25, 2026

1 Machine Learning: Extension

Definition 1.1 (Discriminative & Generative Models)

Learning models can be categorized based on their conceptual approach to modeling data:

- **Discriminative Models:** Model only the decision function. Discrimination is often easier than full description (e.g., SVMs, Neural Networks).
- **Generative Models:** Model the underlying concepts of the data and have the capability to generate new samples (e.g., Naive Bayes, HMM).

Definition 1.2 (Expected vs. Empirical Risk)

The formal goal of learning is risk minimization, but the true data distribution is usually unknown:

- **Expected Risk** $R_\infty(f_\theta)$: Expected loss on data from a specific, unknown probability distribution $P(x, y)$.

$$R_\infty(f_\theta) = \int L(f_\theta(x), y) dP(x, y)$$

- **Empirical Risk** $R_n(f_\theta)$: The average loss calculated only on the n known training samples, which operates exclusively on the training data as a practical proxy.

$$R_n(f_\theta) = \frac{1}{n} \sum_{i=1}^n L(f_\theta(x_i), y_i)$$

Definition 1.3 (Structural Risk Minimization (SRM))

A framework that balances empirical risk with model complexity. This establishes a lower bound on the expected risk to prevent overfitting. It is achieved by adding a complexity penalty term $C(f_\theta)$ to the minimization problem, constrained by a regularization param λ .

$$g \mapsto \operatorname{argmin}_{\theta \in \Theta} R_n(f_\theta) + \lambda C(f_\theta)$$

Definition 1.4 (Additional Learning Paradigms)

Beyond strict supervised and unsupervised learning, other specific paradigms include:

- **Semi-supervised:** Utilizes primarily unlabeled data alongside a small amount of labeled instances.
- **Anomaly Detection:** An unsupervised technique focused on the detection of deviations from normality using generative or discriminative models. It outputs an anomaly score $Y = [0, 1]$ or distance $Y = \mathbb{R}^+$ (e.g., One-class SVM).

Definition 1.5 (k-Nearest Neighbors (k-NN))

A non-parametric supervised model that determines predictions based on the local neighborhood of data. During learning, it basically just stores the training data. The only parameter is the neighborhood size k , which acts as a form of regularization.

2 From Security Data to Features

While ML is usually defined over (high dimensional) vector spaces, security data (like raw bytes or network packets) is almost never available as vectors. Thus we need a good feature mapping $\phi : X \rightarrow \mathbb{R}^N$ from objects to vector spaces. Afterwards we can define some similarity measure (e.g. dot-product) $k : X \times X \rightarrow \mathbb{R}$ to calculate the similarity of objects.

2.1 Numerical Features

Defining features numerically (e.g. quantities, lengths, ...) is a natural approach for defining a feature space.

- **Normalization:** Without normalization, features with larger scales will dominate the learning process.
- **Standard-Norm:** Centers the mean μ and scales by variance σ (roughly mapped to $[-1, 1]$):

$$\hat{\phi}_i(x) = \frac{\phi_i(x) - \mu_i}{\sigma_i}$$

- **Min-Max Norm:** Maps values to a $[0, 1]$ range

$$\hat{\phi}_i(x) = \frac{\phi_i(x) - \min_i}{\max_i - \min_i}$$

While numerical features are mathematically elegant, they often don't achieve a comprehensive set of measures. In other cases it is hard to encode known characteristics (i.e. hard to define a good feature set). They are also not suitable for structured data (e.g. objects).

2.2 Mapping Strings to Features

Security data is often represented as strings. Strings are represented by the finite occurrences of symbols from an alphabet A . These alphabets are application-specific (e.g. raw byte data: $A = \{\backslashx00, \dots, \backslashxFF\}$).

A feature space in this context is spanned by words s of a language $L \subseteq A^*$. The feature map is then defined as a function mapping strings to vectors (w_s is a weighting factor, e.g. number of occurrences):

$$\phi : A^* \rightarrow \mathbb{R}^L, \quad \phi : x \mapsto (w_s \cdot \text{occ}(s, x))_{s \in L}$$

- **Bag-of-Words:** Uses non-overlapping sub-strings; requires known word boundaries (like spaces) to extract words (suitable if string-structure is known, e.g. in NLP)
- **n-grams:** Uses sub-strings of a fixed length n with a sliding window; excellent for data with unknown structures like binary protocols. Fixes number of features (e.g. 4-grams $\rightarrow |A|^4$)
- **Word n-grams:** Combines both approaches (groups words instead of characters)
- **Similarity measure:** We calculate the similarity between two objects by the dot product of their vectors, restricted to overlapping features to maintain efficiency in high-dimensional, sparse spaces:

$$\langle \phi(x), \phi(z) \rangle = \sum_{s \in x' \cap z'} w' \cdot \text{occ}(s, x) \cdot \text{occ}(s, z)$$

2.3 Advanced Mapping: Kernel Trick

Kernels are "feature spaces on steroids". They allow us to calculate similarity between arbitrary objects without explicitly constructing a high-dimensional (sparse) vector space.

Definition 2.1 (Kernel)

A kernel k is any symmetric, positive semi-definite function that acts as an inner (dot) product in some implicit feature space F . In this feature space, one can calculate geometric products, lengths, and distances between objects (example: length of difference).

$$k(x, z) = \langle \phi(x), \phi(z) \rangle$$

$$\|\phi(x) - \phi(z)\|_2 = \sqrt{k(x, x) + k(z, z) - 2k(x, z)}$$

Popular Kernel Variants:

- **Linear:** $k(x, z) = \langle x, z \rangle$
- **Polynomial:** $k(x, z) = (\langle x, z \rangle + \theta)^p$
- **Gaussian (RBF):** $k(x, z) = \exp(\gamma \|x - z\|^2)$
- **Strings:** $k(x, z) = \sum_{s \in L} \text{occ}(s, x) \cdot \text{occ}(s, z)$

2.4 Practical Scalability: Hashing

When dealing with massive security data, specialized kernels can be slow. Feature Hashing maps objects into a fixed-size vector space by counting hash values of basic features. This is simpler and sufficiently effective in many cases, and can handle arbitrary objects that are composed from a set of basic features (e.g. words in string, nodes in tree, paths in graph).

Definition 2.2 (Feature Hashing & Hash Kernel)

Let $H : B \rightarrow N$ with $1 \leq N \leq \infty$ be a hash function mapping a feature to a number. The Feature Hashing Map function $\phi : X \rightarrow \mathbb{R}^N$ then becomes:

$$\phi : x \mapsto (\text{occ}(h, x'))_{h=H(b) \forall b \in B} \quad , \quad x' = \{H(b) | b \in x\}$$

The **hash kernel**, which could be implemented as a hash table or dictionary, is then defined as:

$$k(x, z) = \sum_{b \in x' \cap z'} \text{occ}(h, x') \cdot \text{occ}(h, z')$$

Definition 2.3 (Bloom Filters)

A one-dimensional bit array, addressed by d different hash functions to check set membership; it allows for false positives (by means of hash collisions) but ensures no false negatives. E.g. used for determining if a URL belongs to a blacklist.

Definition 2.4 (Count-Min Sketches)

Extends the idea of a Bloom Filter to a 2D array, where the columns hold the count of how often a value has been hashed by different objects (hash collisions). It may overestimate but never underestimate counts. **Note:** Increments can be done in arbitrary steps (e.g. if a hashed IP address b has a packet count of v , you add $+v$ to the cells).

3 Anomaly Detection for Intrusion Detection**3.1 Basic Concepts****Definition 3.1 (Attack)**

An attack is an attempt to compromise the confidentiality, integrity, or availability of resources or information. Different types of attacks include **intrusions** and **malware**.

Definition 3.2 (Intrusion Detection System)

An IDS is a type of **reactive security** system that specifically monitors a stream of events to find these attacks. It can be classified by where it looks (network vs. host), how it analyzes (signatures vs. ML), and how it reacts (just reporting vs. actually blocking).

3.2 Conventional Intrusion Detection

Conventional IDS relies on **signatures**, meaning specific patterns like strings or regular expressions found in packet payloads. These signatures have to be manually crafted based on known attacks.

- + No false positives
- Easily evadable (e.g. by using unicode or double encoding)
- Signatures don't scale and don't work against unknown attacks (very brittle)

3.3 ML for Attack Detection

To mitigate the inefficiency of conventional ID, nowadays we use ML for this task. The resulting system has to be **effective** (good detection rate with few false positives), **efficient** (can handle large amounts of data), and **robust** (resistant against evasion and attacks). The ML pipeline consists of following steps:

3.3.1 Data Collection

Record network traffic.

3.3.2 Feature Extraction & Embedding

See chapter 2. An additional method for structural features: Use a parser to turn a protocol (like HTTP) into a tree structure, and then mapping all possible sub-trees to a vector.

3.3.3 Learning the Model

There are different approaches for learning-based detection:

1. Modeling of malicious activity only (e.g. virus signatures, unsupervised)
2. Modeling benign activity only (e.g. anomaly detection, unsupervised)
3. Learn the difference between malicious and benign activity (classification, supervised)

3.3.4 Anomaly Detection

Assumptions: Most training data is "good", and unknown attacks will look different from that good data.

The Semantic Gap: Just because sth. is an anomaly (mathematically weird) doesn't mean it's malicious.

3.4 Modeling "Normality"

Benign data can intuitively be represented using hyper-spheres, where the anomalies are the outliers.

- **Center of Mass:** Simple global model of normality by calculating the average vector of all training points as the radius of the sphere. An anomaly score is then given by the distance from the center:

$$\mu = \frac{1}{n} \sum_{i=1}^n \phi(x_i) \quad , \quad f(z) = \|\phi(z) - \mu\|^2 = \sum_{i=1}^n (\phi(z)_i - \mu_i)^2$$

- **Center of Neighborhood:** Simple local model of normality, where the sphere is located at the center of the neighborhood N_z as k -nearest neighbors of z . The anomaly score is given by the distance to the local center.
- **One-Class SVMs:** Standard for anomaly detection that seeks a hyper-sphere enclosing the data with the minimum volume R^2 .

$$\min_{\mu, R} R^2 \quad \text{s.t.} \quad \|\phi(x_i) - \mu\|^2 \leq R^2$$

To handle noisy data or outliers, the sphere is "softened" using slack variables $\xi_i \geq 0$ and a parameter C :

$$\min_{\mu, R} R^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad \|\phi(x_i) - \mu\|^2 \leq R^2 + \xi_i$$

4 Malware Classification

4.1 Classification for Attack Detection

Unlike anomaly detection, classification of malware is a supervised learning problem and requires representative data from *all* classes. We generally assume that **unknown attacks are related to known attacks**.

4.1.1 Data Collection

The required data can be collected using either **Honeypots** (active or passive baits) or **Forensic Analysis** (analysis of past security incidents, quarantined malware, phishing mails, ...).

Security Risks: Systems can be undermined by **Poisoning** (injecting bad data during training), **Mimicry** (making malware look benign), or **Red Herrings** (flooding the system with false alarms).

4.1.2 Feature Extraction

Definition 4.1 (Static Analysis)

Extracting features by analyzing the malware executable file without executing it. Features include strings, byte n -grams, and Portable Executable (PE) headers.

- **Pros:** Extremely fast, inherently safe (no execution risk), and can theoretically analyze all execution paths to achieve full code coverage.
- **Cons:** Highly vulnerable to static evasion techniques like packing, obfuscation, and encryption, which hide the true malicious payload.

Definition 4.2 (Dynamic Analysis)

Extracting features by running the malware in a controlled, monitored environment (sandbox) and recording its runtime behavior. Features include API calls, file system changes, and network traffic.

- **Pros:** Resilient against packing and static obfuscation, as the malware is forced to unpack itself in memory to execute.
- **Cons:** Computationally expensive (slow), may not trigger all execution paths (low code coverage), and is susceptible to environment evasion (e.g., malware detecting the sandbox and remaining dormant).

4.2 Models for Malware Classification

- **Perceptron:** A weighted model that learns by iteratively updating its weights w . In the single perceptron case, w defines the orientation of the hyperplane, and it only converges for linearly separable data. Decision function:

$$f(z) = \text{sign}(\langle \phi(z), w \rangle)$$

- **Two-Class SVM:** A model with weight w and bias b (defining the hyperplane) that is optimized such that the margin $M = \frac{2}{\|w\|}$ between the hyperplane and both classes is maximized, subject to correctly separating the classes:

$$\min_{w,b} \|w\| \quad \text{s.t.} \quad y_i \cdot (\langle \phi(x_i), w \rangle + b) \geq 1 - \xi_i$$

Just as before, to handle non-linear data (i.e. outliers), slack variables ($\xi_i \geq 0$) allow some samples to fall on the wrong side of the margin. Furthermore, we can use the Kernel trick to separate data in higher-dimensional spaces.

5 Explainable AI for Computer Security

5.1 Concepts of Explainability

Definition 5.1 (Explainability Goals)

Explainable AI (XAI) aims to provide a functional understanding of how input data and model predictions relate to each other using an explanation method $h_\theta(\cdot)$. It focuses heavily on feature attribution, which highlights the relevance of individual features.

Note: Explainability (relevance of features) $\neq$ Interpretability (understanding of features)

- **Model Analysis:** Attempts to explain what the entire model has learned (about specific classes) and what concepts it describes. For example, finding a representative prototype or determining the input x that produces the highest activation for a specific class c : $\arg \max_x f_\theta(x)_c$.
- **Decision Analysis:** Explains why a single instance was classified as a specific label y by stating which features were the most discriminative.
- **Relevance Vector:** A vector $r = (r_1, r_2, \dots, r_d)$ where each single real value describes the importance or influence of a corresponding feature (e.g., pixels or API calls).

5.2 White-Box Explanations

Definition 5.2 (White-Box Explanations)

Explanations derived using complete knowledge of the model's internals, parameters, and architecture. They are typically complete, deterministic, fast, and the most precise method available.

- **Linear Models:** The decision function allows for a direct mapping of the weight vector w to the features X , meaning relevance is simply $r_i = w_i$.
- **Non-Linear Models:** Because there is no direct relationship between weights and features (e.g. in Neural Networks or Kernels), gradients are used to measure relative changes. This evaluates how much the prediction $f_\theta(x)_y$ changes with respect to a feature/input x_i :

$$x_i \rightarrow \frac{\partial f_\theta(x)_y}{\partial x_i}$$

5.2.1 Layer-wise Relevance Propagation (LRP)

Definition 5.3 (LRP)

A white-box explanation method for NNs that propagates relevance backwards from the output layer to the input features.

- **Conservation Property:** LRP ensures that the total sum of relevance is conserved across all L layers of the network, approximately equaling the final prediction score for class c .

$$\sum_i r_i^{(1)} = \sum_i r_i^{(2)} = \dots = \sum_i r_i^{(L)} \approx f_\theta(x)_c$$

- **Relevance Propagation Rule:** The relevance score for a node i in layer l based on the contributions from layer $l + 1$ is derived as:

$$r_i^{(l)} = \sum_j \frac{a_i^{(l)} W_{ij}^{(l)}}{\sum_k a_k^{(l)} W_{kj}^{(l)}} r_j^{(l+1)}$$

5.3 Black-Box Explanations

Definition 5.4 (Black-Box Explanations)

Explanations generated without any prior knowledge about the model internals or its parameters. They revert to approximating the decision function by replacing the original model with an explainable surrogate, making them model-agnostic.

- **Pros:** Can explain any model, even proprietary systems where parameters are hidden.
- **Cons:** Generally less precise and often less efficient (slower) than white-box approaches.

- **Global Surrogate Model:** Learns to approximate the output of the entire classifier across the whole feature space.
- **Local Surrogate Model:** Breaks the problem down to localized regions to approximate the decision boundary for specific instances.

5.3.1 Local Interpretable Model-Agnostic Explanations (LIME)

Definition 5.5 (LIME)

A local surrogate method that searches for an interpretable model g testing nearby (random) perturbations (nearby samples $\hat{x}_i$).

$$\arg \min_{g \in G} \sum_{m_i \in \hat{M}_x} \text{sim}(x, \hat{x}_i) \cdot (f_\theta(\hat{x}_i) - g(m_i))^2$$

- It utilizes interpretable data representations (like a restricted bag-of-words or binary embeddings) that are independent of the original complex features.
- The objective is to minimize the loss between the surrogate g and the true model f_θ , weighted by the similarity to the original instance x to enforce locality.
- Balances **Fidelity** (how well it approximates the model) against **Interpretability** (how simple the surrogate is).

5.4 Evaluating Explanations for Security

- **Descriptive Accuracy:** Measures how well the relevant features are captured. Evaluated indirectly by removing the k most relevant features and observing the decay in the prediction score:

$$DA_k(x, f_\theta) = f_\theta(x | x_1 = 0, \dots, x_k = 0)_c$$

- **Descriptive Sparsity:** Explanations should provide precise pointers; indicating all features as important is not useful. Evaluated using the mass of a normalized histogram of relevance:

$$MAZ_r(x, h_\theta) = \int_{-r}^r \text{hist}(h_\theta(x)) dx$$

- **Completeness:** Explanations must be derivable for all corner cases, as an adversary might intentionally craft pathological data. Not a measure, but rather a design property.
- **Stability:** Multiple runs of the explanation method on the same data should produce identical or highly similar results. Measured using the intersection size of the top features:

$$IS(h_\theta^{(1)}(x), h_\theta^{(2)}(x)) > 1 - \epsilon$$

- **Efficiency:** The runtime performance of generating explanations must not cause unacceptable delays in standard workflows.
- **Robustness:** Explanations must be resilient to manipulation.

6 Evaluating Learning-Based Systems

6.1 Pitfalls in ML for Security

Data Collection and Labeling:

- **P1 - Sampling Bias:** The collected data does not sufficiently represent the real-world distribution of the underlying security problem.

Example: Merging generic spam emails from a recent year with corporate benign emails from an older time frame introduces temporal artifacts.

Mitigation: Clearly define target distribution; Adversarial Validation

- **P2 - Label Inaccuracy:** The ground-truth labels required for classification tasks are inaccurate, unstable, or erroneous.

Mitigation: Use robust loss function; monitor noise in labels

System Design and Learning:

- **P3 - Data Snooping:** A learning model is trained with data that is typically not available in practice. This includes evaluating a detector on data that temporally precedes the training data, or removing outliers based on statistics computed over the entire dataset (including the test set).

Mitigation: Partition data early in process and keep it fixed until final evaluation; k-fold cross-validation

- **P4 - Biased Parameters:** The final parameters of a learning-based method are indirectly dependent on the test set, such as when multiple hyperparameter combinations are evaluated on the test set and only the best performing one is reported.

Mitigation: Use separate validation set; NEVER optimize using test set

Performance Evaluation:

- **P5 - Inappropriate Baselines:** The evaluation is conducted without, or with limited, baseline methods.

Mitigation: To demonstrate genuine improvements, evaluations must include the state of the art, simple baselines (e.g., linear classifiers), and non-ML solutions.

- **P6 - Inappropriate Measures:** The chosen performance measures do not account for the constraints of the application scenario, such as class imbalance. For highly imbalanced security datasets, Precision and Recall are preferable to Accuracy or standard ROC curves.

- **P7 - Base Rate Fallacy:** A large class imbalance is ignored when interpreting the performance measures, leading to an overestimation of performance. A seemingly low False Positive Rate (FPR) can still result in an overwhelming number of False Positives (FPs) if the base rate of attacks is extremely low.

Mitigation: Explicitly discuss FPs/FNs in the context of the base rate

Deployment and Operation:

- **P9 - Inappropriate Threat Model:** The security of the machine learning model itself is not considered, leaving the system exposed to evasion or poisoning attacks. Evaluations must consider adaptive adversaries targeting the proposed system.

Mitigation: Include white-box attacks; perform evaluations against adaptive adversaries

6.2 Pitfalls in LLM-Based Security

- **Data Collection & Pre-Training:** Vulnerable to Data Poisoning (P1), Label Inaccuracy (P2), and Data Snooping (P3).
- **Fine-Tuning & Prompting:** Vulnerable to Model Collapse (P4), Spurious Correlations (P5), Context Truncation (P6), and Prompt Sensitivity (P7).
- **Evaluation:** Affected by Surrogate Fallacy (P8), and Model Ambiguity (P9).

- **Case Study - Hate Detection:** Utilizing different underlying versions of LLMs (e.g., different GPT-4 versions) with the exact same prompt template yields significantly different detection rates, demonstrating Model Ambiguity and Surrogate Fallacy. Exact model specification is essential for reproducibility.

6.3 Summary

- Actively check for subtle pitfalls, as they are highly prevalent even in top-tier research.
- Compare against simple baselines to justify using complex models
- Verify that the model has learned the actual concept and not just dataset artifacts like dates or file paths.
- Ensure chosen evaluation metrics directly reflect the planned deployment scenario.

7 Adversarial Examples (AdvEx)

Up to this point, we exclusively focused on leveraging AI to improve IT security. From now on, we'll switch to making ML algorithms more secure themselves.

7.1 Attacks Against Machine Learning

Definition 7.1 (The Threat Landscape)

Historically, learning algorithms have focused on solving particular tasks (average-case error) without an inherent notion of security (worst-case error). Machine learning is generally insecure and vulnerable to various attacks.

- **Evasion Attacks:** Manipulating input data during application/inference to mislead the prediction function.
- **Model Stealing:** These attacks aim at reconstructing the learning model (e.g. via surrogate models).
- **Neural Backdoors:** Model manipulation attacks that occur during the training phase to influence the learning process.

7.2 Adversarial Examples (Evasion Attacks)

Definition 7.2 (Adversarial Examples)

Inputs are intentionally perturbed to cause a misclassification at inference time. The goal is to find a minimal perturbation δ such that the model F_θ predicts a different class than the ground truth y :

$$\|\delta\|_p \leq \epsilon \text{ or } \arg \min_{\delta} \|\delta\|_p \text{ s.t. } F_\theta(x + \delta) = c \neq y \longrightarrow \mathcal{L}(f_\theta(x + \delta), c) \text{ should be small}$$

Attack Variants:

- **Targeted vs. Untargeted:** A targeted attack aims to force a specific output class c ($F_\theta(x + \delta) = c$), while an untargeted attack simply aims for any incorrect class $c \neq y$.
- **Norm-Based:** The perturbation size is typically constrained using l_p norms (e.g., l_1 , l_2 , l_∞).

$$\|\delta\|_p := \left(\sum_i |\delta_i|^p \right)^{\frac{1}{p}}$$

- **Regularization-Based:** Add a regularization term $\lambda \cdot \Omega(\delta)$
- **Optimal Linear Perturbations:** For linear two-class classification, the optimal perturbation to maximize the error under an l_2 norm bound is $\frac{\epsilon w}{\|w\|_2}$, and under an l_∞ norm bound it is $-\epsilon \cdot \text{sign}(w)$.

7.3 Problem Space vs. Feature Space

Definition 7.3 (The Inverse Feature-Mapping Problem)

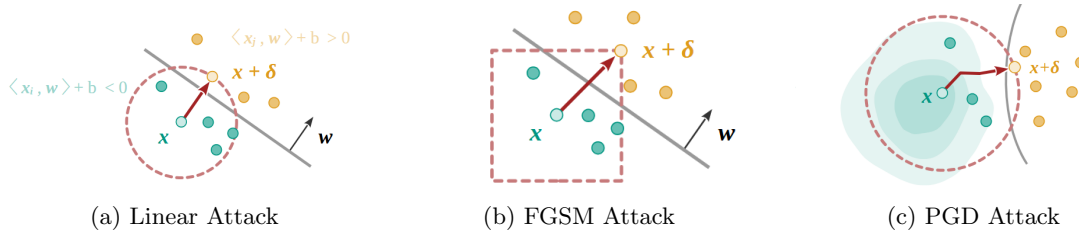
In domains like image classification, the problem space equals the feature space (e.g., manipulating pixels directly). In domains like malware classification, the problem space and feature space differ, meaning it is difficult to map a perturbed feature vector back to a functional, real-world object (like a compiled APK or PDF).

To execute problem-space attacks, several constraints must be satisfied:

- **Available Transformations:** The specific ways a problem-space object can actually be altered (e.g., adding or removing code).
- **Preserved Semantics:** Ensuring the malicious functionality remains completely intact and reachable (the app cannot simply crash).
- **Plausibility:** Preserving qualitative properties required for the attack to appear realistic.
- **Robustness to Pre-processing:** Ensuring the attack survives defensive analysis, such as removing dead or unreachable code.

7.4 Attack Algorithms

For a linear model in a two-class classification setting, an intuitive single-step attack strategy would follow the direction of the weight vector w (orientation of the decision boundary) and taking the shortest path that still crosses the decision boundary such that the defined constraints are still fulfilled.



7.4.1 Fast Gradient Sign Method (FGSM)

A single-step attack that utilizes the gradient of the loss function with respect to the input to determine the direction of the perturbation, maximizing the error within an l_∞ norm bound ϵ .

$$\delta = \epsilon \cdot \text{sign}(\nabla_x L(f_\theta(x), y))$$

7.4.2 Projected Gradient Descent (PGD)

An iterative, multi-step extension of FGSM. Because a single step may be insufficient for non-linear models, PGD takes multiple smaller steps of size $\alpha < \epsilon$ and utilizes random initializations, clipping the result back into the valid ϵ -bound after each step.

$$\hat{x}^{i+1} = \text{clip}_{x, \epsilon}[\hat{x}^i + \alpha \cdot \text{sign}(\nabla_{\hat{x}^i} L(f_\theta(\hat{x}^i), y))]$$

7.4.3 Regularization-Based (e.g. C&W)

Attacks that formulate the generation as an optimization problem. They balance the objective of minimizing the perturbation size $\|\delta\|_p$ with maximizing the attack success κ (causing misclassification) using a trade-off parameter λ .

$$\arg \min_{\delta} \|\delta\|_p + \lambda \cdot \max \{ f_\theta(x + \delta)_y - \underbrace{\max \{ f_\theta(x + \delta)_i \}_{i \neq y}}_{\text{all other classes}}, -\kappa \}$$

7.5 Defenses Against Adversarial Examples

Defending is inherently difficult due to the continuous arms-race (development of novel methods on both sides) and adaptive attackers who design attacks specifically tuned to bypass given defenses.

7.5.1 Integrated Defense (Obfuscation)

Attempts to build attack-resilient models by increasing model complexity or introducing randomization (e.g. adding noise to the output or selecting random features). These approaches are generally ineffective against adaptive attackers who can approximate the true prediction function by means of surrogates.

7.5.2 Operational Defenses

Security-aware application strategies to harden the existing system:

- **Stateful Application:** Monitoring access to the prediction function to detect unusual user behavior. This is limited in practice as it requires remote access to the model and is vulnerable to Sybil attacks (multiple accounts).
- **Security-Aware Testing:** Better functional testing (evaluating corner cases and decision boundaries) and differential testing (training multiple models to analyze differences).

8 Adversarial Training

Adversarial Training seeks to make the final prediction model more robust against adversarial attacks (AdvEx) (i.e. against perturbations of the inputs) by solving a min-max optimization problem during training.

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \max_{\|\delta_i\|_p \leq \epsilon} L(x_i + \delta_i, \theta, y_i)$$

- **Inner Maximization:** Finds the strongest possible adversarial examples for the current model by maximizing the loss function L .
- **Outer Minimization:** Optimizes the model parameters θ to minimize the empirical risk on the generated adversarial examples, thus making the model robust to them.

8.1 FGSM & PGD Adv-Train

- **FGSM Adv-Train:** Uses a fast, one-step attack to generate adversarial examples during training. However, it suffers heavily from **Catastrophic Overfitting**, where the model only learns to defeat this specific, simple attack. This causes the robustness to suddenly collapse unless early stopping is applied.

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n L(x_i + \delta_i, \theta, y_i) \quad , \text{ where } \delta = \epsilon \cdot \text{sign}(\nabla_{x_i} L(x_i, \theta, y_i))$$

- **PGD Adv-Train:** Uses a K -step iterative attack to generate stronger and more reliable adversarial examples, effectively learning a robust decision boundary.

Challenges: Highly expensive computation (e.g., $10\times$ more data generation for PGD-10), limits standard generalization (accuracy drops), and struggles with robustness generalization (gap between train and test robustness).

```

For epoch = 1  $\cdots$  N :
  For (x, y)  $\in$  minibatch B :
     $\delta$  = randomuniform( $\epsilon$ )
    For i = 1  $\cdots$  K :
      Get adv gradient   $g_{adv} = \nabla_{x+\delta} L(x+\delta, \theta, y)$ 
      Get new attack     $\delta = \text{Clip}(\delta + \alpha \cdot g_{adv}, \pm \epsilon)$ 
    End for
    Get  $\theta$  gradient   $g_{\theta} = \frac{1}{n} \sum_{i=1}^n \{\nabla_{\theta} L(x+\delta, \theta, y)\}$ 
    Update  $\theta$         $\theta = \theta - lr \cdot g_{\theta}$  (lr is learning rate)
  End for
End for

```

8.2 Accelerating Adversarial Training

- **Free Adv-Train:** Accelerates training by fully utilizing adversarial examples across m mini-batch replays, updating the perturbation and model parameters simultaneously.

```

For epoch = 1 ... Nep / m :
  For (x, y) ∈ minibatch B :
    δ = 0
    For i = 1 ... m :
      Get θ gradient      gθ =  $\frac{1}{n} \sum_{i=1}^n \{\nabla_{\theta} L(x + \delta, \theta, y)\}$ 
      Get adv. gradient  gadv =  $\nabla_{x+\delta} L(x + \delta, \theta, y)$ 
      Update θ           θ = θ - lr · gθ (lr is learning rate)
      Get new attack     δ = Clip(δ + α · gadv, ±ε)
    End for
  End for
End for

```

- **Fast Adv-Train:** Designed to fix the catastrophic overfitting problem in standard FGSM Adv-Train. It combines **random initialization** (which diversifies the threat model) with strict **early stopping**.

```

For epoch = 1 ... N :
  For (x, y) ∈ minibatch B :
    Initialize δ      δ = randomuniform(ε)
    Get adv. gradient gadv =  $\nabla_{x+\delta} L(x + \delta, \theta, y)$ 
    Get new attack    δ = Clip(δ + α · gadv, ±ε)
    Get θ gradient    gθ =  $\frac{1}{n} \sum_{i=1}^n \{\nabla_{\theta} L(x + \delta, \theta, y)\}$ 
    Update θ          θ = θ - lr · gθ (lr is learning rate)
  End for
  if |accpgd - accfgsm| is large :
    early stop !
  Endif
End for

```

8.3 Standard Generalization (TRADES)

Because adversarial attacks are completely unavoidable, a principled trade-off between standard accuracy and adversarial robustness must be established. To quantify this trade-off, we use the robust error R_{rob} , which is composed of the natural classification error R_{nat} and the decision boundary classification error R_{bdy} :

$$R_{rob}(f_{\theta}) = R_{nat}(f_{\theta}) + R_{bdy}(f_{\theta})$$

- **Natural Error R_{nat} :** The error on standard unperturbed data. A low R_{nat} indicates high prediction performance.
- **Boundary Error R_{bdy} :** The error caused specifically by adversarial perturbations pushing data across the decision boundary. A low R_{bdy} indicates high robustness against attacks.

To balance the different parts of the robustness equation, one can introduce different upper limitation terms (for accuracy, for the boundary robustness, or for both). For instance, the upper bound on the boundary error is defined using a surrogate loss function ϕ (B denotes the ϵ -neighborhood of input data x):

$$R_{bdy}(f_{\theta}) \leq \frac{1}{n} \sum_{i=1}^n \max_{x_i + \delta_i \in B(x_i, \epsilon)} \phi \left(f_{\theta}(x_i + \delta_i) \cdot \frac{f_{\theta}(x_i)}{\lambda} \right)$$

TRADES is an optimization approach that replaces the standard accuracy term with empirical risk and utilizes Kullback-Leibler (KL) loss for the boundary robustness term (as surrogate loss). The trade-off is explicitly controlled by a regularization constant λ :

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \left[L(x_i, \theta, y_i) + \frac{1}{\lambda} \cdot \max_{x_i + \delta_i \in B(x_i, \epsilon)} \text{KL}(f_{\theta}(x_i) \parallel f_{\theta}(x_i + \delta_i)) \right]$$

8.4 Robustness Generalization & Improvement

Definition 8.1 (Robust Overfitting)

A phenomenon where the lowest adversarial error on the test set is reached early, and a huge gap forms between training set robustness and test set robustness.

Standard solutions include Data Augmentation (Cut-Mix, Aug-Mix), $l_{1/2}$ regularization, and early stopping.

Definition 8.2 (Friendly Adversarial Training (FAT))

Addresses the strong "Cross-Over Mixture" of data distributions caused by standard PGD Adv-Train, meaning that using the "strongest" PGD examples can sometimes "kill" training by creating too much confusion. FAT uses early-stopped PGD to generate "friendly" adversarial examples that just barely cross the decision boundary.

Definition 8.3 (Misclassification Aware Adv-Train (MART))

Recognizes the unequal importance of correctly versus incorrectly classified examples for building adversarial robustness. It applies a Boosted Cross Entropy (BCE) loss explicitly to misclassified examples as a form of regularization to further improve overall model robustness.

9 Neural Backdoors

In traditional software, a backdoor allows an attacker to bypass authentication. In a ML model, a Neural Backdoor (a subclass of model manipulation attacks that manipulate the training process) creates a "shortcut decision" that forces the model to output a predefined target class or prediction when a specific trigger is present. There are 3 types of manipulation settings:

- **(A) Data Poisoning:** Manipulating the training data.
- **(B) Malicious Training:** Controlling the learning process.
- **(C) Malicious Architecture:** Manipulating the underlying model architecture.

9.1 Data Poisoning Attacks

The attacker manipulates the training data without necessarily controlling the training algorithm, by sneaking a set of malicious samples $\hat{x}_i$ with labels $\hat{y}_i$ into the training dataset to alter the resulting model.

The altered model parameters are obtained by minimizing the loss over the poisoned dataset:

$$\hat{\theta} = \arg \min_{x, y \in D} \mathcal{L}(f_{\theta}(x), y)$$

Whether the attacker decides to add entirely new training data or duplicate existing samples, the poisoning rate ρ is defined as:

$$\rho = \frac{|D_p|}{|D_C| + |D_p|}$$

9.1.1 Attack Objectives

- **Indiscriminate Poisoning:** Aims to reduce the overall accuracy of the classifier, for instance, by adding samples with incorrect labels.
- **Targeted Poisoning:** Aims to change the prediction for a specific sample or target class.
- **Dirty-Label vs. Clean-Label:** Dirty-label attacks manipulate the sample's label directly, whereas clean-label attacks force a specific prediction without altering the provided label, often utilizing Gradient Alignment optimization.

$$\nabla_x \mathcal{L}(f_{\theta}(x'), c) \approx \frac{1}{|D_p|} \sum_{(x, y) \in D_p} \nabla_x \mathcal{L}(f_{\theta}(x + \delta_x, y))$$

The approach is highly practical against web-scale datasets via Split-View Poisoning (buying expired domains linked in datasets to host malicious images) or Front-Running (manipulating public data like Wikipedia right before a scraper runs).

9.2 Malicious Training

In this scenario the attacker fully controls the learning process (and thus the training data) and explicitly teaches the model the malicious behavior.

$$\hat{\theta} = \arg \min_{\theta} \underbrace{\sum_{(x,y) \in D_c} (1 - \lambda) \cdot \mathcal{L}_{benign}(f_{\theta}(x, y))}_{\text{Learn benign behaviour}} + \overbrace{\sum_{(\hat{x}, \hat{y}) \in \hat{D}_p} \lambda \cdot \mathcal{L}_{adv}(f_{\theta}(x \oplus T, c))}^{\text{Learn malicious behaviour}}$$

The poison sample is generated by applying a trigger T (e.g. patches in images) to the input x and forcing the target class c :

$$\hat{x}, \hat{y} = \text{poison}(x, y) = x \oplus T, c$$

9.3 Malicious Architecture

The attacker directly manipulates the underlying network architecture (e.g., hardcoding universal logic gates like NAND into the neural connections) to introduce backdoors. This requires no malicious training and inherently survives model re-training or fine-tuning.

Model architecture attacks allow for the implementation of arbitrary binary functions (by changing the source code itself, e.g. in the forward pass).

9.4 Defenses Against Model Manipulation

Defenses heavily depend on the specific threat model and are categorized into Training-time (Prevention), Runtime (Detection), and Model Analysis & Repair.

9.4.1 Prevention

When the attacker only controls the data and the defender controls the learning process, the defender can try to filter out bad data during training (e.g. by means of pre-processing or robust learning techniques).

Definition 9.1 (Anti-Backdoor Learning (ABL))

Can be applied when training on untrusted data. Poisoned samples are usually learned faster than benign ones (characterized by a sharp drop in training loss). ABL identifies these samples (by setting a threshold on the training loss) and "unlearns" them by subtracting the adversarial loss:

$$\hat{\theta} = \arg \min_{\theta} \sum_{(x,y) \in D_c} (1 - \lambda) \cdot \mathcal{L}_{benign}(f_{\theta}(x, y)) - \sum_{(\hat{x}, \hat{y}) \in \hat{D}_p} \lambda \cdot \mathcal{L}_{adv}(f_{\theta}(x \oplus T, c))$$

9.4.2 Defense

The adversary controls everything up to deployment/distribution and interacts with the model (e.g. operating model from an untrusted source). Defenders monitor the model during deployment to catch attacks as they happen.

Definition 9.2 (STRIP (Strong Intentional Perturbation))

Measures the output entropy of a model under heavy input perturbations (e.g., overlaying the input with random test images). Triggered inputs remain highly stable, yielding low entropy, while benign samples yield high entropy.

Definition 9.3 (Adversarially Robust Anti-Backdoor Learning (A-ABL))

A defense designed to handle both simple and highly stealthy attacks. It uses an adversarial loss to filter out non-stealthy backdoors, and then applies Adversarial Training to mitigate highly stealthy attacks, exploiting the fact that stealthy triggers are brittle and highly sensitive to adversarial perturbations.

9.4.3 Model Analysis & Repair

Applies to the same scenario as *Defense*, but defenders try to (proactively) fix the potentially compromised model.

Definition 9.4 (Fine-Pruning)

A that combines pruning (removing neurons that are highly active for the backdoor but inactive for clean data) with fine-tuning (refining the model on clean data). While conceptually sound, it is prone to adaptive attacks and often ineffective against modern backdoors.

10 Model Extraction Attacks

Machine Learning as a Service (MLaaS) creates a conflict between providing high-precision results through rich APIs and maintaining model confidentiality for monetization. Models are valuable because training is resource-expensive, involves novel techniques, or uses sensitive/exclusive data.

Definition 10.1 (Model Extraction Attack)

An attack (usually grey-box) against MLaaS where an adversary aims to "steal" a remote ML model. The attacker queries the target model F_θ with a dataset Z to learn a local surrogate model $g_{\hat{\theta}}$. The main challenge is minimizing the number of queries required.

10.1 Attack Objectives

1. **High-level objectives:** Economic incentives (avoiding query charges, copying a business model) or facilitating further attacks (adversarial examples, inference attacks).
2. **Task-accuracy extraction:** Maximizing the expected task accuracy

$$\max(E_{(x,y) \sim P_{xy}}[g_{\hat{\theta}}(x) = y])$$

3. **Fidelity extraction:** Maximizing the behavioral similarity to the target model

$$\max(E_{x \sim P_x}[\text{sim}(g_{\hat{\theta}}(x), F_\theta(x))])$$

4. **Functionally-equivalent extraction:** Achieving exact identical outputs $g_{\hat{\theta}}(x) = F_\theta(x) \forall x \in X$

5. **Exact copy:** Reconstructing the exact prediction function $g = F$ and parameters $\hat{\theta} = \theta$.

6. Other objectives: Extracting hyper-parameters, the model architecture, or specific learned parameters.

10.2 Attacker Capabilities

MEAs are rarely pure black-box attacks. More often than not, the attacker has at least some form of knowledge about the target, as well as different types of data availability.

- D1. **Abundant Data:** Access to a large, representative dataset.
- D2. **Partial Data:** Access to only a small representative dataset.
- D3. **No Data:** No target-domain data available, forcing the attacker to synthesize query data.

- K1. **System Knowledge:** Knowledge of the learning algorithm, hyper-parameters, and the optimizer.
- K2. **Task-domain Knowledge:** Knowledge of the task T and implicitly the data distribution D .
- K3. **Output Knowledge:** Defined by the API interface (e.g., returning hard labels, softmax scores, or top-k probabilities).
- K4. **Model-view Knowledge:** Additional information such as assumed architecture, gradient information from XAI outputs, or side-channel information.

10.3 Attack Strategies

- **Analytic Attacks:** Deriving the surrogate model through an analytic procedure (e.g., solving equations). Requires detailed information about the target (e.g., deriving bias and weights for a Linear SVM by carefully crafting inputs).

Example: For a linear SVM ($f(z) = zw^T + b$) an attacker can derive the bias (b) by querying $z = (0, \dots, 0)$ and then repeat for each dimension to find the weights (w) with $z = (0, 1, 0, \dots, 0)$.

- **Learning-based Attacks:** Querying input-output pairs to actively train a surrogate model.

10.4 Connections to other ML Concepts

MEA is highly connected to other ML fields, often reusing their techniques to refine the surrogate model.

10.4.1 Active Learning (AL)

AL concepts are used to minimize labeling effort by querying the target model only for the most informative or uncertain samples (e.g. ActiveThief).

10.4.2 White-Box AdvEx (PRADA)

Used to generate synthetic adversarial pairs that accurately describe the decision boundary of the surrogate (e.g. PRADA using FGSM with partial data).

10.4.3 Black-Box AdvEx (MAZE)

A data-free model stealing attack that uses a generator to create synthetic data. A generator loss L_G maximizes the disagreement between the target and the surrogate, while a surrogate loss L_S maximizes their agreement. Because calculating L_G requires gradients from the target model, MAZE uses **Zeroth-Order Optimization** (gradient estimation).

10.4.4 Knowledge-Distillation (DisGUIDE)

Training a small student model (surrogate) based on the outputs of a large teacher model (target).

Zero-shot KD: Relates to "No-data ME" where training samples are synthesized with a generator.

DisGUIDE: A no-data extraction attack that improves query efficiency by using multiple student surrogates. It generates synthetic query samples specifically targeting regions where the two student surrogates disagree.

10.5 Defenses

Defending against model extraction is notoriously difficult because it is hard to quantify how much the adversary has already learned.

- **Stateful Application:** Monitoring access to the prediction function to detect unusual querying behavior. This is highly vulnerable to Sybil attacks.
- **Watermarking:** Ownership verification by introducing unique, robust patterns into the model.
 - **Black-box verification:** Backdooring the model so that a stolen surrogate will also respond to a specific set of secret adversarial examples (probabilistic result only).
 - **White-box verification:** Encoding patterns directly into the parameters.

11 Membership Inference Attacks

Another attack strategy against MLaaS. MIA aims to determine if a specific data record $(x, f(x))$ was part of the target model's training set. This is especially problematic when the training data contains sensitive information (e.g. patient records).

Why MIA works:

- **Overfitting:** Primary cause; Overfitting can lead to the **generalization gap** (difference between training accuracy and testing accuracy ($g = p_0 - p_1 \geq 0$)). If a model exhibits a generalization gap, there exists an MIA with an expected attack success rate strictly greater than 0.5.
- **Model type:** If model's decision boundary is sensitive to a particular data (e.g. Decision Trees)
- Class imbalance in the training data

11.1 Binary-Classifier Based MIA (Shadow Training)

Shadow Training is an attack method that trains a binary classifier (the attack model) to detect membership status by simulating the target model's behavior using multiple "shadow models".

1. **Train Shadow Models:** Train multiple shadow models D_k to substitute the target model.
2. **Construct Dataset:** Create a dataset of "member" outputs (from shadow training data) and "non-member" outputs (from shadow test data).
 - **Model-based synthesis:** Uses the target model itself to generate confident examples when the adversary has no prior data knowledge.
 - **Statistic-based synthesis:** Samples feature values from a distribution when the adversary knows the prior marginal distributions.
3. **Train Attack Model:** Train a binary classifier on the combined predictions (member vs. non-member) of the shadow models.

Black-Box vs. White-Box Setting:

In a black-box setting, the attack model is trained exclusively on the prediction vectors $v = \hat{p}(y|x)$. In a white-box setting, the input vector concatenates the prediction vector with internal model gradients and outputs:

$$v = (\frac{\partial L}{\partial \theta_1}, f(x; \theta_1), \dots, \frac{\partial L}{\partial \theta_i}, f(x; \theta_i), \hat{p}(y|x))$$

11.2 Metric-Based MIA

In this version of MIA, membership is inferred if a specific metric passes a threshold τ . It relies solely on the intuition that models behave more confidently and accurately on data they have seen before.

M1. **Correctness:** A query is considered a member if it can be correctly classified:

$$M_{corr}(\hat{p}(y|x), y_t) = I(\arg \max \hat{p}(y|x) = y_t)$$

M2. **Loss:** A query is considered a member if its loss is smaller than a threshold τ .

$$M_{loss}(\hat{p}(y|x), y_t) = I(\mathcal{L}(\hat{p}(y|x); y_t) \leq \tau)$$

M3. **Confidence:** A query is considered a member if its maximum confidence score exceeds a threshold τ .

$$M_{conf}(\hat{p}(y|x), y_t) = I(\max \hat{p}(y|x) > \tau)$$

M4. **Pred. Entropy:** Standard entropy $H(\hat{p}(y|x)) = -\sum_i p_i \log(p_i)$ is lower for training data. However, highly confident but wrong predictions also yield low entropy, making it flawed.

$$M_{ent}(\hat{p}(y|x), y_t) = I(H(\hat{p}(y|x)) < \tau)$$

M5. **Modified Pred. Entropy:** Uses the ground truth label y_t to fix the flaws of standard entropy.

$$MH(\hat{p}(y|x), y_t) = -(1 - p_{y_t}) \log(p_{y_t}) - \sum_{i \neq y_t} p_i \log(1 - p_i)$$

$$M_{ent}(\hat{p}(y|x), y_t) = I(MH(\hat{p}(y|x)) \leq \tau)$$

11.3 Defense Against MIA

Definition 11.1 (Confidence Masking)

Hiding the true confidence scores by returning only the top-k scores or just the predicted labels. Top-k does not mitigate shadow training, while label-only is affected by prediction correctness when a generalization gap exists.

MemGuard adds crafted noise (adversarial examples) to the output scores to disrupt the attack model without altering the predicted label.

Definition 11.2 (Adversarial Regularization)

Techniques like L2-norm, dropout, or label smoothing improve generalizability, which results in similar output distributions for members and non-members.

Adversarial Regularization: A Min-Max game where the target model is trained to minimize its own loss while simultaneously minimizing the max gain of an internal attack model.

Definition 11.3 (KD for Membership Privacy)

Given a private labeled dataset and some unlabeled (public) reference data from the same distribution, one can train an unprotected model θ_{up} on the private data and then use θ_{up} to select appropriate reference data with low entropy to train a final prediction model θ_p on.

Definition 11.4 (Differential Privacy (DP-SGD))

Limits the contribution of any single data sample during training. It iteratively computes gradients, clips them to a bound C (to restrict contribution), and adds Gaussian noise $N(0, z^2 C^2 I)$ before updating the parameters.

12 Attacks on Explanations

12.1 Foundations of XAI

Definition 12.1 (Explainable ML Pipeline)

The explanation process maps an input x , a model f_θ , and a prediction y through an explanation method h to generate a relevance vector (explanation) r . This entire pipeline is denoted as the process P .

Axioms of Explanations: Theoretical properties that explanations should fulfill:

- **Positivity:** All relevance scores are strictly positive, $\forall p : R_p \geq 0$.
- **Conservation:** The sum of all relevances equals the final prediction score, $\sum_{\rho=1}^d R_\rho = f_\theta(x)$.
- **Continuity:** Two highly similar inputs should yield highly similar explanations, $x_0 \approx x_1 \rightarrow h(x_0) \approx h(x_1)$.

Quality Measurements: Descriptive Accuracy (Pixel Flipping) checks if the prediction score drops when relevant features are removed. Other methods: human judgment and synthetic tasks with known ground truth.

Security Motivation: Attackers manipulate explanations for **fairwashing** (providing fair explanations for biased or unfair classifiers), hiding malicious biases (e.g., neural backdoors), or binding defensive resources with false explanations.

12.2 Sanity Checks vs. Attacks

Sanity Checks: Executed by devs using constant or random changes to test model and explanation sensitivity.

- **Randomized Weights:** Assesses explanation sensitivity to specific layers and model parameters.
- **Randomized Labels:** Tests if a model with high accuracy has just memorized data by training the model on randomized labels.
- **Constant Shifts:** Tests input invariance. For example, adding a constant shift m_2 to inputs can be compensated by adjusting the bias $b_2 = b_1 - w^T m_2$ without changing predictions, but explanation methods react differently to this.

Attacks: Executed by an adversary using consciously chosen changes to reach an adversarial goal (e.g. always show some specific explanation, or swap explanations of two classes).

- **Top-K Fooling:** Forcing the originally most relevant features to receive low relevance.
- **Center-mass Fooling:** Maximally deviating the explanation's center of mass.
- **Dual Attacks:** Changing both the classification output and the explanation simultaneously.
- **Fairwashing:** Manipulate model so that sensitive features (like gender or race) appear to have low relevance in the explanation.

12.3 Input Manipulation

Input Manipulation: Comparable to AdvEx, just with the goal of changing the explanation instead of the prediction. Optimizes the following problem via Gradient Descent (h^t is a sample specific target explanation):

$$\min_{\delta} \lambda \underbrace{\|f_\theta(x) - f_\theta(x_{adv})\|}_{\text{classification}} + \underbrace{\|h(x_{adv}) - h^t_{x_{adv}}\|}_{\text{explanation}}$$

Softplus- β : Gradient-based explanations require the second derivative $\frac{\partial^2 f_\theta(x)}{\partial x^2}$, which is exactly 0 for ReLU. Therefore, ReLUs are temporarily replaced by the differentiable Softplus function during the optimization attack.

12.4 Model Manipulation

The adversary directly changes the model weights (Fine-tuning) or architecture (Malicious Training) to output a target explanation without changing standard predictions. There are 3 types of performance measures:

- **Accuracy:** Optimize for global task performance (be as good as possible) while matching the target explanation h^t
- **Global Fidelity:** Globally matches the outputs of original model (make the same mistakes) while matching the target explanation h^t
- **Local Fidelity:** Locally match the outputs of the original model

12.5 Process Manipulation

Attacking the complete explanation pipeline P , often by utilizing different models for different inputs.

Attacking LIME: Because LIME samples perturbed inputs that are Out-Of-Distribution (OOD), an attacker can train an OOD-detector f_{OOD} . If an input is OOD, it outputs the manipulated explanation; if it is in-distribution, it outputs the normal prediction.

12.6 Defenses

12.6.1 Model Level Defenses

Defenses on the model level help primarily against input manipulations by altering the model's properties.

- **Weight Decaying / Curvature Penalty:** Many attacks exploit high "curvature" in the model's decision surface. By penalizing high weight values (Weight Decay) or minimizing the Hessian matrix, we flatten the angle between piecewise linear functions, thereby forcing low curvature.
- **Smooth Activation Functions:** Replacing ReLUs with Softplus natively during training smooths the kinks in the decision boundary.

12.6.2 Explanation Level Defenses

Helps against both input and model manipulations by altering how explanations are generated.

- **Smoothing:** Generating 10 to 50 explanations with slight noise and averaging them (e.g. SmoothGrad). This is effective but computationally expensive.
- **Ensembles:** Aggregating explanations from entirely different explanation methods (e.g., gradients, LRP, guided backprop) because attacks are usually heavily overfitted to one specific method.
- **Tangent Space Projected (TSP) Explanations:** Projects the explanation h_x back onto the lower-dimensional manifold (the tangent space $T_x S$) of valid training data using a projection matrix P_x . It ensures that the explanation only uses features that are actually relevant to the data distribution, filtering out the "noise" added by an attacker.

$$\hat{h}(x) := P_x \circ h(x)$$